🧠 LangGraph Quiz

📌 Section A: MCQs (1 mark each)

1. What is LangGraph primarily used for?

a) Data visualization

b) Building graph-based AI workflows

c) Database management

d) Web scraping

Answer: b) Building graph-based AI workflows

2. In LangGraph, nodes represent:

a) Database tables

b) Functions or steps in a workflow

c) UI components

d) API keys

Answer: b) Functions or steps in a workflow

3. What is the main advantage of graph-based workflows?

a) Linear execution only

b) No need for state

c) Flexible branching and looping

d) Faster internet speed

Answer: c) Flexible branching and looping

4. What does “state” represent in LangGraph?

a) Server location

b) Memory shared across nodes

c) Programming language

d) UI design

Answer: b) Memory shared across nodes

5. Which component decides the next step in a LangGraph workflow?

a) Tokenizer

b) Edge

c) Prompt

d) Dataset

Answer: b) Edge

6. A multi-step agent is best described as:

a) A single API call

b) A system executing multiple connected steps

c) A database query

d) A static webpage

Answer: b) A system executing multiple connected steps

7. What is the purpose of conditional edges in LangGraph?

a) To delete nodes

b) To control flow based on conditions

c) To store data

d) To train models

Answer: b) To control flow based on conditions

8. Which of the following is required to define a graph in LangGraph?

a) Nodes and edges

b) Only APIs

c) Only datasets

d) Only UI components

Answer: a) Nodes and edges

9. StateGraph in LangGraph is used for:

a) Styling UI

b) Managing workflow state and transitions

c) Creating databases

d) Sending emails

Answer: b) Managing workflow state and transitions

10. What is the output of a node in LangGraph?

a) Only text

b) Updated state

c) Images only

d) HTML code

Answer: b) Updated state

📌 Section B: Short Answer (2–3 marks each)

11. Explain the concept of graph-based workflows in LangGraph.

Answer: Graph-based workflows use nodes and edges to represent steps and transitions, allowing flexible execution with branching and looping.

12. What is state management and why is it important in multi-step agents?

Answer: State management maintains shared data across steps, ensuring continuity and coordination in multi-step workflows.

13. Differentiate between nodes and edges in LangGraph.

Answer: Nodes are steps/functions, while edges define how control flows between them.

14. What is a StateGraph and how does it help in building agents?

Answer: StateGraph manages both workflow and state, enabling structured and context-aware agent execution.

15. Give one real-world example where a multi-step agent can be useful.

Answer: An AI email assistant that drafts, reviews, and sends emails.

📌 Section C: Coding-Based Questions (4–5 marks each)

16. Consider the following code:

```
def step1(state):
    return {"msg": "Hello " + state["name"]}
```

Answer: The function takes input state and returns an updated state with a greeting message using the name value.

17. Write a simple LangGraph workflow with two steps:

· Step 1: Take user input

· Step 2: Generate a response

Answer:

Step1: input = user input

Step2: response = LLM(input)

Return response

18. How would you add a conditional step in a LangGraph workflow? Explain with logic.

Answer: Use conditional edges. If condition true → go to node A, else → go to node B.

📌 Section D: Case Study (5–8 marks)

19. Design a multi-step agent using LangGraph for a "Student Assistant Bot" that:

· Takes a question

· Checks if it is about assignments or exams

· Routes to the correct handler

· Returns a response

Answer:

Nodes: input node, classifier node, assignment handler, exam handler, response node

Edges: input → classifier → conditional routing → handler → response

State: stores question, category, and response

📌 Section E: Advanced Conceptual (5 marks)

20. Compare traditional linear AI pipelines with graph-based agent workflows. Why is LangGraph better for complex systems?

Answer: Linear pipelines are rigid and sequential, while graph workflows allow branching, looping, and dynamic control, making LangGraph better for complex systems.